



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

Proyecto Antispam

Curso: Calidad y Pruebas de Software

Docente: Patrick Cuadros Quiroga

Integrantes:

Mamani Cori, Cristhian Carlos

(2023077282)

Jahuira Pilco, Dayan Elvis

(2022075749)

**Tacna – Perú
2026**



CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	CM - DJ	PCQ	PCQ	26/04/2026	Versión Original
2.0	CM - DJ	PCQ	PCQ	29/04/2026	Versión 2.0
3.0	CM - DJ	PCQ	PCQ	06/07/2026	Versión Final

Sistema Antispam Documento de Visión

Versión 3.0

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	CM - DJ	PCQ	PCQ	26/04/2026	Versión Original
2.0	CM - DJ	PCQ	PCQ	29/04/2026	Versión 2.0
3.0	CM - DJ	PCQ	PCQ	06/07/2026	Versión Final

ÍNDICE GENERAL

1. Introducción	4
1.1. Propósito	4
1.2. Alcance	4
1.3. Definiciones, Siglas y Abreviaturas	6
1.4. Referencias	6
1.5. Visión General	6
2. Posicionamiento	7
2.1. Oportunidad de negocio	7
2.2. Definición del problema	8
3. Descripción de los interesados y usuarios	9
3.1. Resumen de los interesados	9
3.2. Resumen de los usuarios	9
3.3. Entorno de usuario	10
3.4. Perfiles de los interesados	10
3.5. Perfiles de los Usuarios	10
3.6. Necesidades de los interesados y usuarios	11
4. Vista General del Producto	11
4.1. Perspectiva del producto	11
4.2. Resumen de capacidades	11
4.3. Suposiciones y dependencias	12
4.4. Costos y precios	12
4.5. Licenciamiento e instalación	12
4.6. Gestión de Documentación (GitHub Wikis)	12
4.7. Plan de Desarrollo (Roadmap)	13
5. Características del producto	13
6. Restricciones	14
7. Rangos de calidad	14
8. Precedencia y Prioridad	15
9. Otros requerimientos del producto	15
a) Estandares legales	15
b) Estandares de comunicación	16
c) Estandares de cumplimiento de la plataforma	16
d) Estandares de calidad y seguridad	17
10. CONCLUSIONES	17
11. RECOMENDACIONES	18



12. BIBLIOGRAFIA

13. WEBGRAFÍA



19

19

Informe de Visión

1. Introducción

1.1. Propósito

El propósito de este documento es definir la visión general del proyecto Aegis Filter, estableciendo sus objetivos, alcance, características principales y la planificación para su desarrollo. Sirve como un acuerdo fundamental entre el equipo de desarrollo y los evaluadores para comprender qué problema se está resolviendo y cómo se medirá el éxito del sistema.

1.2. Alcance

Aegis Filter es una aplicación web backend desarrollada en Laravel que implementa un servicio automatizado para la filtración de comentarios tipo Spam. El alcance del proyecto abarca la programación del motor de reglas, el aprovisionamiento de infraestructura en la nube (Microsoft Azure) mediante Terraform, la orquestación de la aplicación y base de datos (MySQL 8) con Docker, y la implementación de un flujo completo de Integración y Despliegue Continuo (CI/CD) utilizando GitHub Actions, SonarCloud y Snyk.

1.3. Definiciones, Siglas y Abreviaturas

Término	Definición
Spam	Mensajes no solicitados, repetitivos o con enlaces maliciosos enviados en plataformas web.
CI/CD	Integración Continua y Despliegue Continuo (prácticas DevOps).
IaC	Infraestructura como Código (uso de Terraform

	para gestionar servidores).
VM	Máquina Virtual (servidor alojado en Azure).

1.4. Referencias

- Rúbrica de evaluación del curso Calidad y Pruebas de Software.
- Documentación oficial de Laravel 11 y PHP 8.2.
- Documentación técnica de Microsoft Azure, Terraform y Docker.

1.5. Visión General

El documento detalla la problemática actual de la moderación manual de comentarios y cómo Aegis Filter se posiciona como una solución tecnológica automatizada y escalable. A continuación, se describen los perfiles de los usuarios, las capacidades clave del sistema y se delinean las métricas de calidad y restricciones tecnológicas exigidas.

2. Posicionamiento

2.1. Oportunidad de negocio

La gestión y moderación manual de comentarios en plataformas web consume excesivo tiempo y recursos. Aegis Filter ofrece una oportunidad para optimizar los procesos operativos al automatizar la detección de spam con un alto grado de precisión. Esto permite a los administradores enfocarse en tareas de mayor valor, reduciendo costos operativos y mejorando la seguridad de la información. Al estar basado en la nube bajo una arquitectura de contenedores, el producto es altamente escalable.

2.2. Definición del problema

- El problema de: la publicación masiva de comentarios fraudulentos, enlaces maliciosos y ataques de bots de spam.
- Afecta a: administradores de sitios web, moderadores de contenido y la comunidad de usuarios finales.
- Cuyo impacto es: la degradación de la calidad del contenido, aumento de riesgos de seguridad (phishing), pérdida de credibilidad de la plataforma y un incremento innecesario en los costos operativos de moderación manual.
- Una solución exitosa sería: implementar un servicio backend automatizado que intercepte, evalúe (mediante expresiones regulares y listas negras) y bloquee automáticamente el contenido malicioso antes de ser almacenado en la base de datos de producción.

3. Descripción de los interesados y usuarios

3.1. Resumen de los interesados

- Administradores de Plataformas Web y E-commerce: Dueños de sitios que buscan proteger su sección de comentarios o reseñas de enlaces maliciosos y estafas sin perder tiempo en revisiones manuales.
- Equipos de Operaciones y DevOps: Profesionales interesados en una infraestructura moderna y reproducible, que valoran que el sistema esté dockerizado y se despliegue automáticamente mediante Terraform en Azure.

- Usuarios Finales y Comunidad Digital: Personas que interactúan en la plataforma y se benefician de un entorno seguro, limpio de spam y libre de riesgos de phishing o contenido fraudulento.

3.2. Resumen de los usuarios

- Administrador de Sistemas / DevOps: Perfil con acceso total. Encargado de configurar el servidor en Azure, gestionar los contenedores Docker y monitorear la salud del pipeline CI/CD en GitHub.
- Moderador: Usuario del panel de control web. Su función es visualizar las métricas de spam bloqueado y gestionar posibles falsos positivos que el filtro intercepte.

3.3. Entorno de usuario

El sistema opera 100% en la nube. Los administradores interactúan con la infraestructura de forma remota a través de conexiones seguras (SSH) al servidor Linux (Debian 12) alojado en Azure. La moderación del sistema se realiza a través de cualquier navegador web moderno (Chrome, Edge, Firefox).

3.4. Perfiles de los interesados

Buscan la demostración de un producto funcional, estable y seguro que evidencie la correcta aplicación de métricas de calidad de software y prácticas modernas de despliegue continuo.

3.5. Perfiles de los Usuarios

- El usuario administrador debe poseer sólidos conocimientos técnicos en herramientas DevOps, redes y administración de servidores Linux.
- El usuario moderador requiere únicamente habilidades ofimáticas y de navegación web básicas.

3.6. Necesidades de los interesados y usuarios

- Automatización: Reducir drásticamente la intervención manual en el filtrado de datos.
- Transparencia: Disponer de registros (logs) claros sobre los motivos por los cuales un mensaje fue catalogado como Spam.
- Disponibilidad: El sistema debe mantenerse activo, estable y accesible desde internet.

4. Vista General del Producto

4.1. Perspectiva del producto

Aegis Filter funciona como un servicio intermedio (Middleware/Service) integrado dentro del ecosistema de una aplicación web desarrollada en Laravel. El sistema recibe las peticiones HTTP con los comentarios de los usuarios, las procesa a través de un motor de reglas y expresiones regulares, y decide autónomamente si permite su

almacenamiento en la base de datos (MySQL 8) o si las rechaza como contenido malicioso.

4.2. Resumen de capacidades

- Filtrado automatizado de comentarios mediante la detección de palabras prohibidas (listas negras) y patrones de URL sospechosos (Expresiones Regulares).
- Aprovisionamiento de infraestructura 100% automatizado y reproducible mediante scripts de Terraform (IaC).
- Despliegue contenedorizado, modular e inmutable utilizando Docker y Docker Compose.
- Análisis de código y vulnerabilidades automatizado en cada actualización de código mediante SonarCloud y Snyk.

4.3. Suposiciones y dependencias

El sistema tiene una dependencia estricta de la disponibilidad de los servicios de GitHub (Actions, Repositories) y Docker Hub para el correcto funcionamiento del pipeline CI/CD.

4.4. Costos y precios

De acuerdo con la Factibilidad, el costo total valorizado del desarrollo del proyecto es de S/ 3,675.00. Operativamente, el mantenimiento de la infraestructura en la nube (Azure VM, IP pública y almacenamiento) asciende a S/ 75.00 mensuales.

4.5. Licenciamiento e instalación

El código fuente de la aplicación ha sido desarrollado utilizando herramientas de código abierto (Open Source) y no requiere el pago de licencias privativas. La instalación en el entorno de producción está completamente automatizada mediante el archivo de flujo de trabajo de GitHub Actions (ci-cd.yml).

4.6. Gestión de Documentación (GitHub Wikis)

Para garantizar un mantenimiento sostenible del proyecto, toda la documentación técnica se gestiona a través de GitHub Wikis. Esta wiki funciona como la “Única Fuente de Verdad”, albergando guías de instalación, comandos de Terraform y diccionarios de base de datos, facilitando la comprensión del sistema para evaluadores y futuros desarrolladores sin necesidad de inspeccionar directamente el código fuente.

4.7. Plan de Desarrollo (Roadmap)

La gestión ágil y planificación del proyecto se ha estructurado utilizando GitHub Projects (Roadmap), dividiendo el ciclo de vida en 4 hitos (Milestones):

1. Lógica Base (Laravel y Servicios).
2. Infraestructura como Código (Terraform en Azure).
3. Contenedorización y CI/CD (Docker y Actions).
4. Calidad y Seguridad (Integración con SonarCloud y Snyk).

5. Características del producto

- Pipeline de Despliegue Seguro: Ningún código llega a producción si no aprueba satisfactoriamente las pruebas de calidad (PHPUnit) en el entorno de integración.
- Infraestructura Elástica: Capacidad de escalar verticalmente los recursos del servidor en Azure modificando únicamente una variable en el archivo de Terraform.
- Reglas Configurables: El motor Anti-Spam permite añadir fácilmente nuevas reglas de detección sin alterar la lógica central del servicio.

6. Restricciones

- Recursos del Servidor: El entorno de producción en Azure debe garantizar un mínimo de 1.5 GB de memoria RAM libre para que los contenedores de Laravel y MySQL operen sin caídas.
- Compatibilidad de Lenguaje: El código fuente está estrictamente restringido para ser ejecutado en entornos con PHP 8.2 o superior, utilizando el framework Laravel 11.

7. Rangos de calidad

- **Cobertura de Código:** Se establece como rango de calidad un mínimo del 75% de Code Coverage en las pruebas unitarias reportadas por SonarQube/SonarCloud.
- **Seguridad y Vulnerabilidades:** El reporte de Snyk en el pipeline de CI/CD debe arrojar 0 (cero) vulnerabilidades de nivel Alto o Crítico en las dependencias de Composer y NPM.

8. Precedencia y Prioridad

El orden lógico de ejecución para garantizar la integridad del sistema es:

1. Aprovisionamiento de la infraestructura en la nube (Terraform).
2. Ejecución exitosa de las pruebas lógicas del software (PHPUnit).
3. Construcción (Build) de la imagen del contenedor Docker.
4. Despliegue (Deploy) en el servidor de Azure.

9. Otros requerimientos del producto

a) Estándares legales

El proyecto simula el manejo de datos de usuarios, por lo que su arquitectura está diseñada respetando buenas prácticas de privacidad, en concordancia referencial con la Ley de Protección de Datos Personales (Ley N° 29733), evitando almacenar contraseñas o datos sensibles en texto plano.

b) Estándares de comunicación

La comunicación hacia el servidor está restringida por un Grupo de Seguridad de Red (NSG) en Azure, permitiendo únicamente el tráfico web estándar (Puertos 80 y 443) y administración remota segura y cifrada vía SSH (Puerto 22).

c) Estándares de cumplimiento de la plataforma

El sistema operativo del servidor de producción debe ser una distribución Linux compatible con las imágenes de Docker utilizadas, priorizando Debian 12 (Bookworm) por su estabilidad.

d) Estándares de calidad y seguridad

Se aplican enfoques de Desarrollo Guiado por Pruebas (TDD) para el módulo SpamFilterService y se integra una cultura de

DevSecOps al auditar cada cambio de código en busca de fallos de seguridad (Snyk) y deuda técnica (SonarCloud).

10. CONCLUSIONES

Se demuestra que el proyecto Aegis Filter está rígidamente estructurado bajo estándares profesionales de la industria del software. La combinación de servicios web (Laravel), infraestructura automatizada (Terraform) y prácticas de Integración Continua asegura que el sistema no solo resuelva el problema del spam, sino que sea mantenible, seguro y escalable. Asimismo, el uso de GitHub Wikis y Roadmap evidencia una gestión de proyecto colaborativa y madura.

11. RECOMENDACIONES

Se recomienda mantener estrictamente actualizadas las páginas de la Wiki en GitHub cada vez que se modifique la arquitectura de infraestructura o se añadan nuevas variables de entorno.

Se sugiere actualizar el Roadmap del proyecto para contemplar futuras integraciones, como el uso de APIs externas de Inteligencia Artificial para un análisis semántico más profundo de los comentarios, complementando al filtro actual.

12. BIBLIOGRAFIA

Microsoft Azure. (2026). Documentación oficial de máquinas virtuales e infraestructura de Azure.

HashiCorp. (2026). Terraform Registry: Azure Provider Documentation.

Laravel. (2026). Laravel 11.x Documentation: Testing and Services.

Docker Inc. (2026). Docker Compose reference.

13. WEBGRAFIA

Guías de GitHub Actions y Wikis: <https://docs.github.com/es>

Documentación de calidad de código en SonarCloud:

<https://docs.sonarsource.com/sonarcloud/>

Auditoría de seguridad con Snyk: <https://docs.snyk.io/>